

Лабораторная работа № 4

Операционные системы — НПИбд-01-25

Взваров Михаил

В отчёте представлены результаты выполнения лабораторной работы № 4 по дисциплине «Операционные системы».

Приведены ход выполнения работы, полученные результаты и выводы.

Содержание

1	Лабораторная работа №4	6
1.1	Продвинутое использование git	6

Список иллюстраций

Список таблиц

1 Лабораторная работа №4

1.1 Продвинутое использование git

1.1.1 Цель работы

Получение навыков правильной работы с репозиториями git, освоение Gitflow Workflow, Conventional Commits и семантического версионирования.

1.1.2 Задание

1. Установить git-flow, Node.js, pnpm.
 2. Установить commitizen и standard-changelog.
 3. Создать репозиторий на GitHub.
 4. Настроить Conventional Commits.
 5. Инициализировать git-flow.
 6. Создать релиз 1.0.0.
 7. Создать feature-ветку.
 8. Создать релиз 1.2.3.
-

1.1.3 Выполнение работы

1.1.3.1 1. Установка программного обеспечения

Были установлены следующие пакеты:

```
'''bash sudo dnf copr enable elegos/gitflow sudo dnf install -y gitflow sudo dnf  
install -y nodejs sudo dnf install -y pnpm ##### 2. Настройка Node.js и установка  
инструментов
```

Была выполнена настройка pnpm и установка глобальных пакетов:

```
'''bash pnpm setup source ~/.bashrc pnpm add -g commitizen pnpm add -g  
standard-changelog
```

1.1.3.2 3. Создание репозитория на GitHub

На GitHub был создан публичный репозиторий с именем „git-extended“.

1.1.3.3 4. Инициализация локального репозитория

На локальной машине был создан и настроен репозиторий:

```
'''bash mkdir git-extended cd git-extended git init echo «# git-extended» >  
README.md git add README.md git commit -m «first commit» git remote add  
origin git@github.com:mikhailvzvarov/git-extended.git git push -u origin master
```

1.1.3.4 5. Настройка Conventional Commits

Был создан файл „package.json“ для настройки commitizen:

```
'''json { «name»: «git-extended», «version»: «1.0.0», «description»: «Git repo for  
educational purposes», «main»: «index.js», «repository»: «git@github.com:mikhailvzvarov/git-  
extended.git», «author»: «Mikhail Vzvarov (1032251972@rudn.ru)», «license»:  
«CC-BY-4.0», «config»: { «commitizen»: { «path»: «cz-conventional-changelog» } } }
```

1.1.3.5 6. Инициализация git-flow

Выполнена инициализация git-flow:

```
' „bash git flow init | Параметр | Значение | |----|----| | Production releases  
| 'master“ | | Development | „develop“ | | Feature branches | „feature/“ | | Release  
branches | „release/“ | | Hotfix branches | „hotfix/“ | | Version tag prefix | „v“ | git  
branch -set-upstream-to=origin/develop develop
```

1.1.3.6 7. Создание релиза 1.0.0

Был создан первый релиз проекта:

```
'''bash git flow release start 1.0.0 standard-changelog --first-release git add  
CHANGELOG.md git commit -m «chore(site): add changelog» git flow release finish  
1.0.0 -m «Release 1.0.0» git push --all git push --tags gh release create v1.0.0 -F  
CHANGELOG.md
```

1.1.3.7 8. Работа с feature-веткой

Была создана ветка для разработки новой функциональности:

```
'''bash git flow feature start feature_branch После завершения работы над  
функцией ветка была объединена с develop: bash  
git flow feature finish feature_branch
```

1.1.3.8 9. Создание релиза 1.2.3

Был создан новый релиз с версией 1.2.3:

```
'''bash git flow release start 1.2.3 Обновлено версия в package.json: bash  
nano package.json bash  
git add package.json git commit -m «chore(package): bump version to 1.2.3»
```

Обновлён CHANGELOG: bash

```
standard-changelog
```

Добавлен обновлённый CHANGELOG: bash

```
git add CHANGELOG.md git commit -m «chore(site): update changelog»
```

Завершён релиз: bash

```
git flow release finish 1.2.3 -m «Release 1.2.3»
```

Отправлены изменения в удалённый репозиторий: bash

```
git push --all git push --tags
```

Создан релиз на GitHub: bash

```
gh release create v1.2.3 -F CHANGELOG.md
```

1.1.4 Домашнее задание (Ответы на вопросы)

1.1.4.1 1. Что такое Gitflow Workflow?

Ответ: Gitflow Workflow — это модель ветвления для Git, предложенная Винсентом Дриссенем. Она предполагает использование двух основных веток: „master“ (для релизов) и „develop“ (для разработки), а также вспомогательных веток: „feature“, „release“ и „hotfix“.

Последовательность работы по модели Gitflow:

- Из ветки „master“ создаётся ветка „develop“
 - Из ветки „develop“ создаются ветки „feature“
 - После завершения работы ветка „feature“ сливается с „develop“
 - Из „develop“ создаётся ветка „release“
 - После завершения подготовки релиз сливается с „master“ и „develop“
 - Если в „master“ обнаружена проблема, создаётся ветка „hotfix“
 - После исправления „hotfix“ сливается с „master“ и „develop“
-

1.1.4.2 2. Какие основные ветки используются в Gitflow?

Ответ:

- **„master“** (или „main“) — содержит официальную историю релизов. В этой ветке всегда находится стабильная версия кода, готовая к развёртыванию.
 - **„develop“** — основная ветка разработки, в которой объединяются все новые функции. Из этой ветки создаются релизные ветки.
-

1.1.4.3 3. Для чего нужны ветки feature?

Ответ: Ветки „feature“ создаются для разработки новых функций. Они создаются от „develop“ и после завершения работы сливаются обратно в „develop“. Это позволяет:

- Разрабатывать несколько функций параллельно
 - Не нарушать стабильность основной ветки
 - Удобно управлять изменениями
-

1.1.4.4 4. Для чего нужны ветки release?

Ответ: Ветки „release“ создаются для подготовки нового релиза. Они позволяют:

- Завершить разработку и исправить мелкие ошибки
 - Создать документацию перед выпуском
 - Изолировать подготовку релиза от разработки новых функций
-

1.1.4.5 5. Что такое семантическое версионирование (SemVer)?

Ответ: SemVer — это система версионирования в формате „МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ“:

- **МАЖОРНАЯ** — увеличивается при несовместимых изменениях API
 - **МИНОРНАЯ** — увеличивается при добавлении новой функциональности (совместимой)
 - **ПАТЧ** — увеличивается при обратно совместимых исправлениях ошибок
-

1.1.4.6 6. Что такое Conventional Commits?

Ответ: Conventional Commits — это спецификация, определяющая структуру сообщений коммитов: ### Выводы

В ходе выполнения лабораторной работы я:

1. Установил и настроил программное обеспечение:

- git-flow — для реализации модели ветвления Gitflow
- Node.js и npm — для работы с инструментами Conventional Commits
- commitizen — для оформления коммитов по стандарту
- standard-changelog — для генерации журнала изменений

2. Освоил работу с Conventional Commits через „git cz“:

- Научился выбирать типы коммитов
- Оформлять сообщения по стандарту
- Соблюдать структуру коммитов

3. Изучил Gitflow Workflow:

- Создал релиз 1.0.0 с генерацией CHANGELOG
- Создал и завершил feature-ветку
- Создал релиз 1.2.3 с обновлением версии

4. Научился создавать релизы на GitHub через утилиту „gh“:

- Создание релизов с автоматическим описанием из CHANGELOG.md
- Публикация тегов и веток в удалённом репозитории

Цель работы полностью достигнута. Полученные навыки будут полезны при организации рабочего процесса над проектами в команде, а также при управлении версиями и релизами программного обеспечения. ### Библиография

1. Gitflow Workflow. — <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
2. Semantic Versioning. — <https://semver.org/>
3. Conventional Commits. — <https://www.conventionalcommits.org/>
4. standard-changelog. — <https://github.com/conventional-changelog/conventional-changelog>
5. Git Documentation. — <https://git-scm.com/doc>